

## Final Report

### Final Report Instructions

#### Overview

The final report is the main written deliverable of the project. It must be self-contained: a reader who has not seen your milestone reports should still be able to understand what you built, how you parallelized it, how well it runs, and why.

Build on the work from Milestones 1–4, but add new analysis: a performance model and a scalability study. Where it fits, connect your results to topics from lecture, e.g., GPU memory hierarchy, coalesced accesses, roofline model, MPI collectives, Amdahl/Gustafson, isoefficiency.

Every project must include a working **multi-GPU** implementation that combines **CUDA** and **MPI**. You may run your experiments on the teaching cluster or on your own hardware (lab cluster, cloud credits, etc); document the hardware used in the report.

#### Deliverables

You will turn in two items as a group:

1. **Code.** The MPI + CUDA implementation of your algorithm, together with the C++ wrapper code (host-side driver, I/O, any baseline code, etc) needed to build, run, and test it on multiple ranks.
2. **Final report.** The report has a **maximum length of 6 pages**, excluding references and figures. Larger groups are expected to produce more work and a deeper analysis (more algorithmic variants explored, more thorough profiling, etc.). The format is single-spaced, 11pt font, 1-inch margins.

You may include an appendix with extra material (additional plots, tables, derivations, code listings, etc.) beyond the page limit. Graders are not required to read the appendix when grading, so any content that is essential to your argument must appear in the main body of the report.

#### Report Content

Your final report should address the following:

1. **Project description.** State the computational problem you are solving. Describe the inputs, outputs, and any assumptions or simplifications. Explain why this problem matters and why multi-GPU parallelism is needed.
2. **Algorithms and state of the art.** Describe the core algorithms you implemented. Survey the state of the art and the main algorithmic variants that are applicable to your problem, citing relevant papers, libraries, or open-source implementations. Explain which variant you chose and how it compares to existing implementations.
3. **Parallelization strategy (CUDA + MPI).** Describe the difficulties you encountered when parallelizing the computation and the strategies you considered. Specify how the problem is decomposed across MPI ranks, what data each rank owns, and what must be exchanged at each step. Describe

your GPU kernels: thread/block organization, use of shared memory, coalesced accesses, etc. Describe your communication pattern: which MPI collectives or point-to-point operations you use. Justify your design choices.

4. **Performance model.** Propose a quantitative model that predicts how your implementation should perform. Depending on your project, this can include arithmetic intensity and a roofline analysis, a communication cost model ( $\alpha + \beta n$ ), an Amdahl/Gustafson analysis, an isoefficiency analysis, or any combination of these. State your assumptions. The model is what you will use to interpret your measurements. It must be specific enough to make falsifiable predictions.
5. **Benchmarking and instrumentation.** Describe how you instrumented and measured your code, for example using CUDA events, Nsight Systems, Nsight Compute, MPI profiling. Report the metrics you collected. Compare the measurements to the predictions of your performance model and explain any discrepancies.
6. **Bottleneck analysis.** Identify what currently limits your performance. Is your application compute-bound, memory-bandwidth-bound, latency-bound, limited by kernel-launch overhead, host-device transfers, load imbalance, etc? Tie your observations back to the hardware (GPU memory hierarchy, warp execution model, interconnect topology) and to material from lecture.
7. **Algorithmic variants and their effect on performance.** Explore at least one non-trivial algorithmic or implementation variant (e.g., tiling sizes, problem decompositions, kernel fusion). Quantify the improvement and explain how each variant changes the bottleneck.
8. **Scalability analysis.** Study how performance depends on problem size and on the number of GPUs/ranks. Include a strong scaling study (fixed total problem size, increasing rank count) and a weak scaling study (fixed problem size per rank, increasing rank count). Report speedup and parallel efficiency in clearly labeled figures.
9. **Correctness and verification.** Describe how you verified that your code is correct: comparison against a CPU reference, analytical solutions, published results, multi-rank consistency checks (e.g., 1, 2, 4 ranks), regression tests. Report the accuracy achieved (e.g., maximum/relative error).
10. **Discussion, limitations, and future work.** Reflect on the trade-offs you made. What would you do differently with more time or more hardware? Which optimizations gave the largest payoff, and which were not worth the complexity?

## Computing Resources

The teaching cluster is a shared resource. Individual jobs should run for at most a few minutes, and **never more than 15 minutes**. Use small problem sizes for development and debugging, and reserve larger runs for your final benchmarking and scaling study. If you use your own hardware (lab cluster, cloud), document the hardware and software environment in the report.

## Grading Criteria

Your final report will be evaluated as follows (weights are tentative and subject to adjustment):

| Category                                                 | Weight |
|----------------------------------------------------------|--------|
| Parallelization (CUDA + MPI design and implementation)   | 20%    |
| Performance model, benchmarking, and bottleneck analysis | 20%    |
| Scalability study and effect of algorithmic variants     | 15%    |
| Correctness, testing, and verification                   | 10%    |
| Writing quality, organization, and clarity of discussion | 15%    |
| Visual quality of plots and figures                      | 10%    |
| Depth of analysis and connection to lecture material     | 10%    |

Larger groups should go deeper in each category.

### Additional Guidance

Clarity is one of the most important qualities of a strong report. The report should be self-contained and readable by someone who has not seen your milestone submissions, so assume the reader is not familiar with your specific problem or implementation.

Begin with the problem itself: state the inputs, the outputs, the computation being performed, and what makes it challenging. Explain why the problem matters and why parallel computing is necessary in this setting. Only then describe how your parallel implementation addresses the challenge.

The main body of the report should emphasize results and analysis, not implementation details. While the CUDA kernels, MPI communication, memory layout, and low-level optimizations likely consumed most of your development effort, the report should not read as an implementation log. Focus on the performance measurements, the scalability behavior, the bottlenecks you identified, and the interpretation of your results. The central question is not only what you built, but what you learned from building and benchmarking it.

Support your conclusions with figures and tables. If kernel-level details, derivations, profiling outputs, or auxiliary experiments are too technical or too long for the main text, place them in an appendix. The graders are not required to read the appendix, so anything essential to your argument must appear in the main body.

### Submission

Submit your code and your final report on Gradescope as a group. Include all group members in the submission. The report has a maximum length of 6 pages regardless of group size (excluding references, figures, and appendices), single-spaced, 11pt font, 1-inch margins. This is a maximum, not a target.